WR CLICKER

Jeu de type Clicker — Application Web PHP/MySQL

Cas d'utilisation · BTS SIO SLAM · 30/03/2026

CU-01 S'inscrire et créer un compte

Acteurs principaux : Joueur

Acteur secondaire : Système (base de données MySQL)

Déclencheur : Le joueur clique sur « 📝 Inscription » depuis la page d'accueil (index.php).

Préconditions

- L'application est démarrée et accessible via un navigateur.
- Le serveur PHP et la base de données MySQL sont opérationnels (Laragon).
- Le joueur ne possède pas encore de compte WR Clicker.

Postconditions (garanties en cas de succès)

- Un nouveau compte est créé dans la table joueur de la base wr_clicker.
- Le joueur est redirigé vers vue_confirmation_inscription.php avec le nombre de lignes insérées.
- Le joueur peut désormais se connecter avec son pseudonyme et son mot de passe.

Scénario nominal

1. Le joueur arrive sur index.php et clique sur le lien « 📝 Inscription ».
2. La page vue_formulaire.php s'affiche avec un formulaire à trois champs : Nom d'utilisateur (nickname), Adresse Email et Mot de passe.
3. Le joueur remplit les trois champs et clique sur « 🚀 Créer mon compte ».
4. Le navigateur envoie une requête POST vers traitement.php.
5. traitement.php vérifie que le champ nickname n'est pas vide.
6. Il appelle la fonction insert_membre($Email, $nickname, $mdp) de fonctions.php.
7. insert_membre() exécute INSERT INTO joueur (Email_utilisateur, nickname, mdp_utilisateur) VALUES (...).
8. La base retourne 1 ligne affectée ; insert_membre() renvoie 1.
9. traitement.php redirige vers vue_confirmation_inscription.php?nb=1.
10. La page de confirmation affiche : « ✅ Les informations ont bien été enregistrées ! ».

Scénarios alternatifs / exceptions

- 5A – Le champ nickname est vide : traitement.php redirige vers vue_erreur.php avec le message « ATTENTION : Le champ nom ou prénom n'a pas été rempli correctement ».
- 7A – Le nickname ou l'e-mail existe déjà en base (doublon) : MySQL lève une erreur ; insert_membre() affiche le message d'erreur et arrête l'exécution via die().
- 7B – La connexion à la base est impossible (param_connexion.php invalide) : mysqli_connect() échoue silencieusement ; la requête retourne false et le joueur est redirigé vers vue_erreur.php.

CU-02 Se connecter et accéder au jeu

Acteurs principaux : Joueur

Acteur secondaire : Système (base de données MySQL)

Déclencheur : Le joueur clique sur « 🔑 Connexion » depuis index.php ou depuis vue_erreur.php.

Préconditions

- L'application est démarrée et accessible via un navigateur.
- Le joueur possède un compte existant (CU-01 réalisé avec succès).
- Aucune session PHP active n'est ouverte.

Postconditions (garanties en cas de succès)

- Une session PHP est ouverte avec $_SESSION['nickname'] et $_SESSION['Id_Joueur'] renseignés.
- Le score, la progression (croissance, croissance2) et le dernier bonus du joueur sont chargés depuis la base.
- Le joueur est redirigé automatiquement vers jeu.php après 3 secondes.

Scénario nominal

1. Le joueur arrive sur vue_connexion.php et saisit son pseudonyme (nickname) et son mot de passe.
2. Il clique sur « ✅ Se connecter ».
3. Le navigateur envoie une requête POST vers traitement_connexion.php.
4. traitement_connexion.php vérifie que les deux champs ne sont pas vides.
5. Il appelle la fonction connexion($nickname, $mdp) de fonctions.php.
6. connexion() exécute SELECT * FROM joueur WHERE nickname=... AND mdp_utilisateur=....
7. La base retourne 1 ligne : les identifiants sont corrects.
8. connexion() retourne le tableau complet du joueur (Id_Joueur, nickname, score, etc.).
9. traitement_connexion.php démarre la session et stocke $_SESSION['nickname'] et $_SESSION['Id_Joueur'].
10. Il redirige vers vue_confirmation.php.
11. vue_confirmation.php affiche « Vous vous êtes bien connecté » et redirige automatiquement vers jeu.php après 3 secondes.
12. jeu.php charge depuis la base le score sauvegardé, les prix des bonus, la progression et le dernier bonus via les fonctions obtenir_score(), obtenir_bonus_joueur(), obtenir_progression() et obtenir_dernierbonus().
13. Ces données sont injectées dans des variables JavaScript ; la session de jeu démarre avec l'état sauvegardé.

Scénarios alternatifs / exceptions

- 4A – L'un des champs est vide : traitement_connexion.php redirige vers vue_erreur.php avec le message « ATTENTION : L'un des champs n'a pas été rempli correctement ».
- 6A – Les identifiants ne correspondent à aucun enregistrement en base : connexion() retourne false ; traitement_connexion.php redirige vers vue_erreur.php avec le message « Utilisateur inexistant ».
- 12A – Le joueur tente d'accéder directement à jeu.php sans session ouverte : la vérification isset($_SESSION['nickname']) échoue ; il est redirigé vers vue_erreur.php.

CU-03 Jouer et acheter des bonus

Acteurs principaux : Joueur

Acteur secondaire : Aucun (logique entièrement côté client — JavaScript)

Déclencheur : Le joueur est sur jeu.php après une connexion réussie (CU-02).

Préconditions

- Le joueur est connecté et sur jeu.php.
- Les variables JavaScript (score, croissance, croissance2, prixbonus1…5, dernierbonus) sont initialisées depuis les données PHP injectées à l'affichage de la page.

Postconditions (garanties en cas de succès)

- Le score est mis à jour en temps réel dans l'affichage (élément #score).
- Les effets du ou des bonus achetés sont appliqués immédiatement (croissance et/ou croissance2 modifiés).
- Les prix des bonus achetés sont augmentés selon leur coefficient et affichés sur les boutons.
- Si un nouveau bonus plus avancé a été acheté, l'image de la voiture change et un son est joué.

Scénario nominal

1. Le joueur clique sur la voiture (input type image, id='voiture') : la fonction JavaScript clic() est appelée.
2. clic() incrémente score de croissance2 km et met à jour l'affichage (#score) ainsi que le champ caché #sauvegarde.
3. Lorsque le joueur a suffisamment de km, il clique sur un bouton de bonus (ex. : « 🏁 25km »).
4. Le bouton appelle la fonction bonus1() (ou bonus2()…bonus5() selon le bouton).
5. bonus1() vérifie que score >= prixbonus1.
6. Si la condition est remplie : croissance += 3, score -= prixbonus1, prixbonus1 *= 2. (les valeurs peuvent différer selon le bonus acheté)
7. L'affichage du score et du prix du bouton sont mis à jour.
8. Les champs cachés du formulaire de sauvegarde (hidden_prixbonus1, hidden_croissance) sont mis à jour.
9. Si ce bonus est le plus avancé jamais acheté (dernierbonus < 1) : l'image de la voiture change (images/bonus1.jpg) et un son est joué (audio/bonus1.mp3).
10. La production passive s'active (si elle n'est pas déjà active) : passif() démarre un setInterval qui ajoute croissance km chaque seconde.

Scénarios alternatifs / exceptions

- 5A – Le joueur n'a pas assez de km pour acheter le bonus : la condition score >= prixbonus est fausse ; un alert() JavaScript s'affiche : « Vous n'avez pas assez de score. » Aucune modification n'est apportée.
- 10A – La production passive est déjà active (gain == true) : passif() ne crée pas de second setInterval, évitant une accélération non voulue.

CU-04 Sauvegarder sa progression

Acteurs principaux : Joueur

Acteur secondaire : Système (base de données MySQL)

Déclencheur : Le joueur clique sur le bouton « 💾 Sauvegarder » depuis jeu.php.

Préconditions

- Le joueur est connecté et sur jeu.php (session $_SESSION['nickname'] et $_SESSION['Id_Joueur'] actives).
- Une session de jeu est en cours : le joueur a un score en mémoire côté client.

Postconditions (garanties en cas de succès)

- Le score du joueur est mis à jour dans la colonne score de la table joueur.
- Les prix actuels des 5 bonus sont sauvegardés ou mis à jour dans la table appartenir.
- Les valeurs croissance, croissance2 et dernierbonus sont mises à jour dans la table joueur.
- Le joueur est redirigé vers vue_confirmation_sauvegarde.php confirmant la sauvegarde.
- À la prochaine connexion, le jeu reprendra depuis cet état sauvegardé.

Scénario nominal

1. Le joueur clique sur « 💾 Sauvegarder ».
2. Le navigateur soumet le formulaire masqué (id='formSauvegarde') en POST vers traitement_sauvegarde.php. Les champs cachés contiennent : score, prixbonus1 à 5, croissance, croissance2, dernierbonus.
3. traitement_sauvegarde.php récupère $_SESSION['nickname'] et $_SESSION['Id_Joueur'].
4. Il vérifie que le champ score est présent et non vide.
5. Il appelle insert_sauvegarde($score, $nickname) : UPDATE joueur SET score='$score' WHERE nickname='$nickname'.
6. Il appelle sauvegarder_bonus($Id_Joueur, $Id_Bonus, $prix_bonus) pour chacun des 5 bonus : INSERT INTO appartenir … ON DUPLICATE KEY UPDATE prix_bonus.
7. Il appelle sauvegarder_progression($Id_Joueur, $croissance, $croissance2) : UPDATE joueur SET croissance, croissance2.
8. Il appelle sauvegarder_dernierbonus($Id_Joueur, $dernierbonus) : UPDATE joueur SET dernierbonus.
9. insert_sauvegarde() retourne 1 ligne affectée.
10. traitement_sauvegarde.php redirige vers vue_confirmation_sauvegarde.php?nb=1.
11. La page affiche : « ✅ Les valeurs ont bien été sauvegardées. 1 ligne(s) mise(s) à jour. »

Scénarios alternatifs / exceptions

- 4A – Le champ score est absent ou vide dans le POST : traitement_sauvegarde.php redirige vers vue_erreur.php avec le message « ATTENTION : Il y a eu un problème lors de la sauvegarde, veuillez réessayer ».

- 5A – La requête UPDATE échoue (connexion BDD perdue) : insert_sauvegarde() retourne false ; traitement_sauvegarde.php redirige vers vue_erreur.php avec « Erreur lors de l'insertion des données ».
- 9A – Le score n'a pas changé depuis la dernière sauvegarde : MySQL ne compte aucune ligne modifiée (0 ligne affectée) ; insert_sauvegarde() retourne 0 ; traitement_sauvegarde.php redirige vers vue_erreur.php.